UNIT - IV

Transactions and Concurrency Control-Introduction, Transactions, Nested Transactions, Locks, Optimistic concurrency control, Timestamp ordering. Distributed Transactions-Introduction, Flat and Nested Distributed Transactions, Atomic commit protocols, Concurrency control in distributed transactions, Distributed deadlocks, Transaction recovery.

Transactions and Concurrency Control:

Introduction

Banking transaction for a customer (e.g., at ATM or browser)

Transfer $100 from saving to checking account;

Transfer $200 from money-market to checking account;

Withdraw $400 from checking account.

Transaction (invoked at client): /* Every step is an RPC */

1. savings.withdraw(100) /* includes verification */
2. checking.deposit(100) /* depends on success of 1 */
3. mnymkt.withdraw(200) /* includes verification */
4. checking. deposit(200) /* depends on success of 3 */
5. checking.withdraw(400) /* includes verification */
6. dispense(400)
7. commit

- All the following are RPCs from a client to the server
- Transaction calls that can be made at a client, and return values from the server:

Transactions

openTransaction() -> trans;

starts a new transaction and delivers a unique transaction identifier (TID) trans. This TID will be used in the other

operations in the transaction.

closeTransaction(trans) -> (commit, abort);

ends a transaction: a commit return value indicates that the transaction has committed;

an abort return value indicates that

it has aborted.

abortTransaction(trans);

aborts the transaction.

- TID can be passed implicitly (for other operations between open and close) with CORBA
- deposit(amount)
- deposit amount in the account
- withdraw(amount)
- withdraw amount from the account
- getBalance() -> amount
- return the balance of the account
- setBalance(amount)
- set the balance of the account to amount
- Sequence of operations that forms a single step, transforming the server data from one consistent state to another.
    - All or nothing principle: a transaction either completes successfully, and the effects are recorded in the objects, or it has no effect at all. (even with multiple clients, or crashes)
- A transactions is indivisible (atomic) from the point of view of other transactions
    - No access to intermediate results/states of other transactions
    - Free from interference by operations of other transactions

But…

- Transactions could run concurrently, i.e., with multiple clients
- Transactions may be distributed, i.e., across multiple servers

- Atomicity: All or nothing
- Consistency: if the server starts in a consistent state, the transaction ends the server in a consistent state.
- Isolation: Each transaction must be performed without interference from other transactions, i.e., the non-final
- effects of a transaction must not be visible to other transactions.
- Durability: After a transaction has completed successfully, all its effects are saved in permanent storage.
- Atomicity: store tentative object updates (for later undo/redo) – many different ways of doing this
- Durability: store entire results of transactions (all updated objects) to recover from permanent server crashes.
- The effect of an operation refers to
- The value of an object set by a write operation
- The result returned by a read operation.
- Two operations are said to be conflicting operations, if their combined effect depends on the order they are executed, e.g., read-write, write-read, write-write (all on same variables). NOT read-read, NOT on different variables.
- Two transactions are serially equivalent if and only if all pairs of conflicting operations (pair containing one operation from each transaction) are executed in the same order (transaction order) for all objects (data) they both access.
- Why? Can start from original operation sequence and swap the order of non-conflicting operations to obtain a series of operations where one transaction finishes completely before the second transaction starts

- Why is the above result important? Because: Serial equivalence is the basis for concurrency control protocols for transactions.

Operations of different transactions		*Conflict*	*Reason*
read	*read*	No	Because the effect of a pair of *read* operations does not depend on the order in which they are executed
read	*write*	Yes	Because the effect of a *read* and a *write* operation depends on the order of their execution
write	*write*	Yes	Because the effect of a pair of *write* operations depends on the order of their execution

Concurrency control

• **Lost update**

– 3 accounts (A, B, C)

» with balances 100, 200, 300

– T1 transfers from A to B, for 10% increase

– T2 transfers from C to B, for 10% increase

– Both T1, T2 read balance of B (200)

– T1 overwrites the update by T2

» Without seeing it

• **Inconsistent retrievals**

– T1: transfers 10% of account A to account B

– T2: computes sum of account balances

– T2 computes sum before T1 updates B

Recoverability from aborts

- Servers must prevent a aborting Tx from affecting other concurrent Tx's.
  - Dirty reads:
    - » T2 sees result update by T1 on account A
    - » T2 performs its own update on A & then commits.
    - » T1 aborts -> T2 has seen a "transient" value
      - T2 is not recoverable
    - » If T2 delays its commit until T1's outcome is resolved:
      - Abort(T1) -> Abort(T2)
      - However, if T3 has seen results of T2:
        - Abort(T2) -> Abort(T3) !

- **Premature writes:**
  - Assume server implements abort by maintaining the "before" image of all update operations
    - » T1 & T2 both updates account A
    - » T1 completes its work before T2
    - » If T1 commits & T2 aborts, the balance of A is correct
    - » If T1 aborts & T2 commits, the "before" image that is restored corresponds to the balance of A before T2
    - » If both T1 & T2 abort, the "before" image that is restored corresponds to the balance of A as set by T1
- Tx's should delay both their reads & updates in order to avoid interference
  - Strict execution -> enforce isolation
- Servers should maintain tentative versions of objects in volatile memory
- Tx's should delay both their reads & updates in order to avoid interference
  - Strict execution -> enforce isolation
- Servers should maintain tentative versions of objects in volatile memory

Concurrency Control: Locks

- Transactions:
  - Must be scheduled so that their effect on shared data is serially equivalent

– Two types of approach

» Pessimistic If something can go wrong, it will

Operations are synchronized before they are carried out

» **Optimistic** In general, nothing will go wrong

Operations are carried out, synchronization at the end of the transaction

– Locks (pessimistic)

» can be used to ensuring serializability

» lock(x), unlock(x)

– Oldest and most widely used CC algorithm

– A process before read/write requests the scheduler to grant a lock

– Upon finishing read/write the lock is released

– In order to ensure serialized transaction Two Phase Locking (2PL) is used

– How Locks prevent consistency problems

- Lost update and inconsistent retrieval:
- Causes:

  » are caused by the conflict between ri(x) and wj(x)

  » two transactions read a value and use it to compute new value
- Prevention:

  » delay the reads of later transactions until the earlier ones have completed

  » Disadvantage of Locking
- Deadlocks

– Strict 2PL avoids Cascading Aborts

- A situation where a committed transaction has to be undone because it saw a file it shouldn't have seen

– Problems of Locking

- Deadlocks
- Livelocks

  » A transaction can't proceed for an indefinite amount of time while other transactions continue normally. It happens due to unfair locking.

- Lock overhead
  - » If the system doesn't allow shared access--wastage of resources
- Avoidance of Cascading Aborts may be costly
  - » Strict 2PL in fact, reduces the effect of concurrency

– Transaction managers (on server side) set locks on objects they need. A concurrent trans. cannot access locked objects.

– Two phase locking:

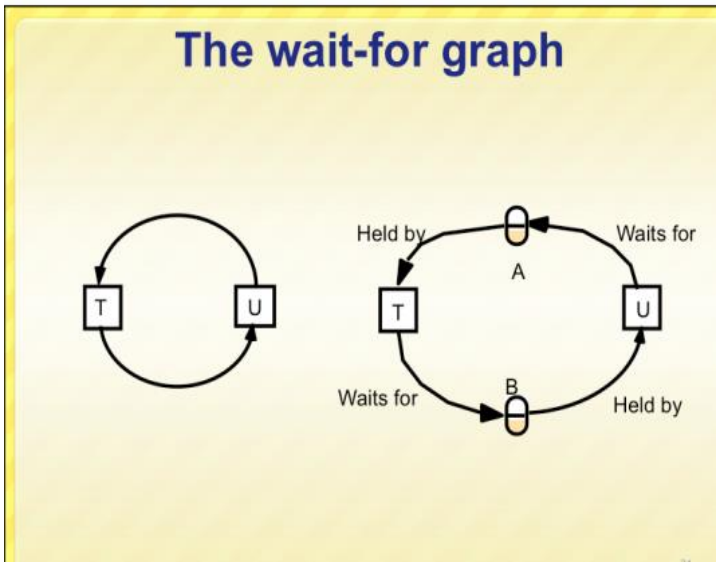
- In the first (growing) phase of the transaction, new locks are only acquired, and in the second (shrinking) phase, locks are only released.
- A transaction is not allowed acquire any new locks, once it has released any one lock

Deadlock with write locks

Transaction *T*		Transaction *U*	
Operations	**Locks**	**Operations**	**Locks**
a.deposit(100);	write lock *A*		
		b.deposit(200)	write lock *B*
b.withdraw(100)			
•••	waits for *U*'s lock on *B*	*a.withdraw(200);*	waits for *T*'s lock on *A*
•••		•••	
•••		•••	
		•••	

Transaction T locks A and waits for Uto release the lock on B, U on the other hand locks B and waits for T to release the lock on A.

→Circular hold and wait → dead lock



- **Necessary conditions for deadlocks**
  - Non-shareable resources (exclusive lock modes)
  - No preemption on locks
  - Hold & Wait or Circular Wait

Resolution of deadlock

Transaction T		Transaction U	
Operations	**Locks**	**Operations**	**Locks**
a.deposit(100);	write lock *A*		
		b.deposit(200)	write lock *B*
b.withdraw(100)			
•••	waits for *U*'s lock on *B*	*a.withdraw(200);*	waits for T's lock on *A*
	(timeout elapses)	•••	
T's lock on *A* becomes vulnerable, unlock *A*, abort T		•••	
		a.withdraw(200);	write locks *A* unlock *A*, *B*

Strategies to Fight Deadlock

- Lock timeout (costly and open to false positives)
- Deadlock Prevention: violate one of the necessary conditions for deadlock (from 2 slides ago), e.g., lock all objects before transaction starts, aborting entire transaction if any fails
- Deadlock Avoidance: Have transactions declare max resources they will request, but allow them to lock at any time (Banker's algorithm)
- Deadlock Detection: detect cycles in the wait-for graph, and then abort one or more of the transactions in cycle
- We have seen locking has some problems
- OCC based on the following simple idea:

Distributed Transactions

In previous chapter, we discussed transactions accessed objects at a single server. In the general case, a transaction will access objects located in different computers. Distributed transaction accesses objects managed by multiple servers.

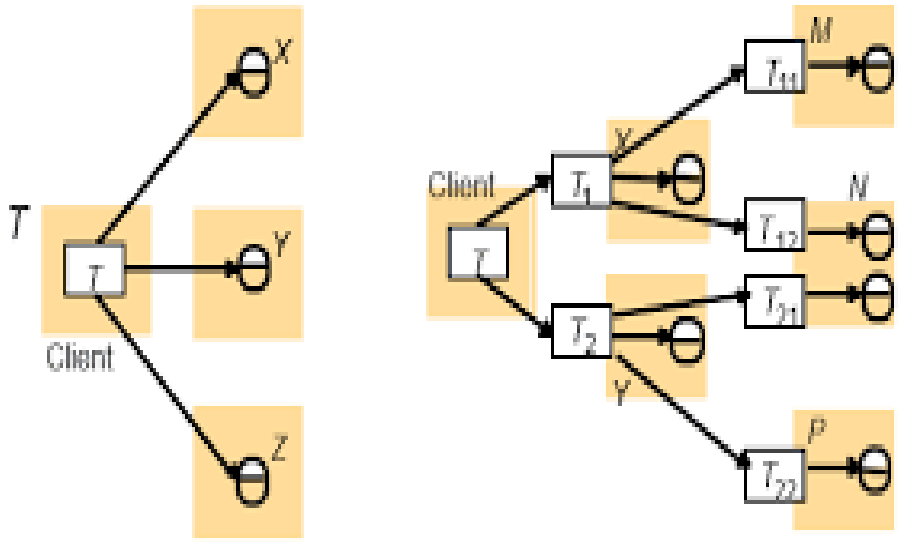
The atomicity property requires that either all of the servers involved in the same transaction commit the transaction or all of them abort. Agreement among servers are necessary.

Transaction recovery is to ensure that all objects are recoverable. The values of the objects reflect all changes made by committed transactions and none of those made by aborted ones.

Flat and Nested Distributed Transaction

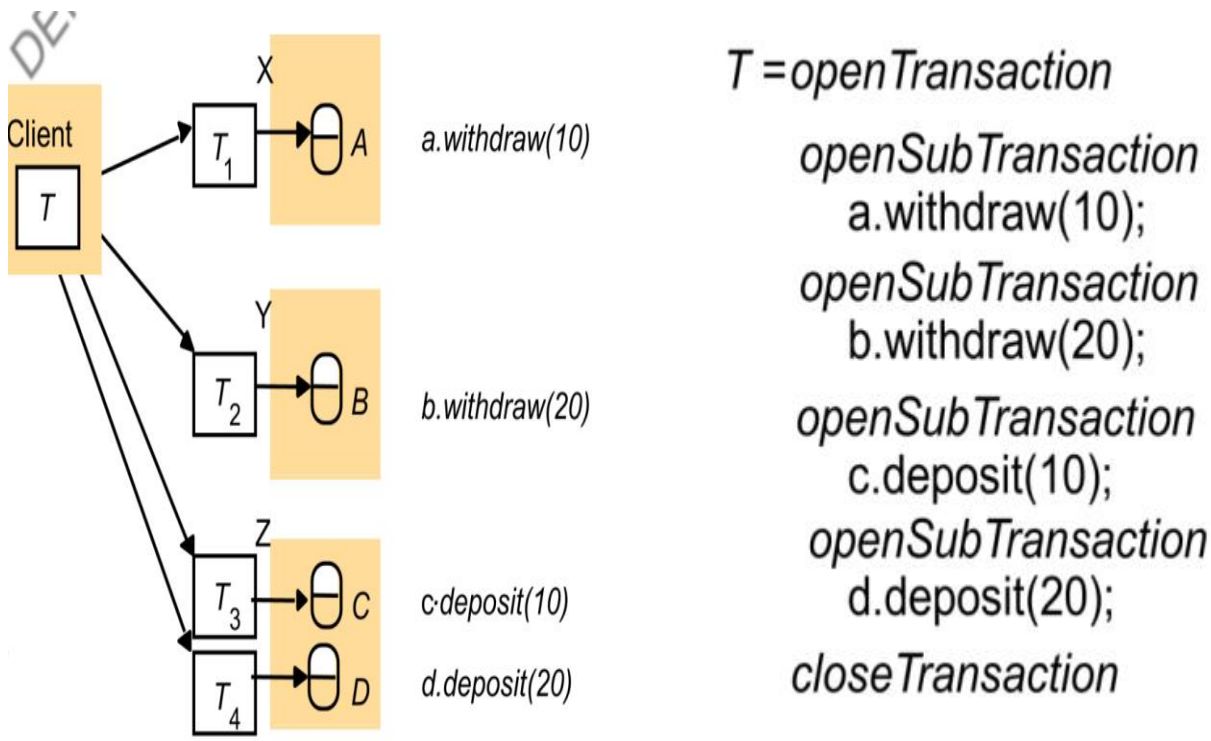
- In flat transaction, a client makes requests to more than one server.
- It completes each of its requests before going on to next one, accessing server objects sequentially.

- In nested transaction, the top-level open sub transactions; each of which can further open sub transactions.
- The sub transactions at same level run concurrently.



Flat transaction

Nested transaction



Atomic Commit Protocol

In the **first phase**, each participant votes for the transaction to be committed or aborted. Once voted to commit, not allowed to abort it. So before votes to commit, it must ensure that it will eventually be able to carry out its part, even if it fails and is replaced.

A participant is said to be in a prepared state if it will eventually be able to commit it. So each participant needs to save the altered objects in the permanent storage device together with its status-prepared.

Phase 1 (voting phase):

1. The coordinator sends a can Commit?. request to each of the participants in the transaction.
2. When a participant receives a can Commit? request it to reply with its vote (Yes or No) to the coordinator.

Before voting Yes, it prepares to commit by saving objects in permanent storage. If the vote is No the participant aborts immediately.

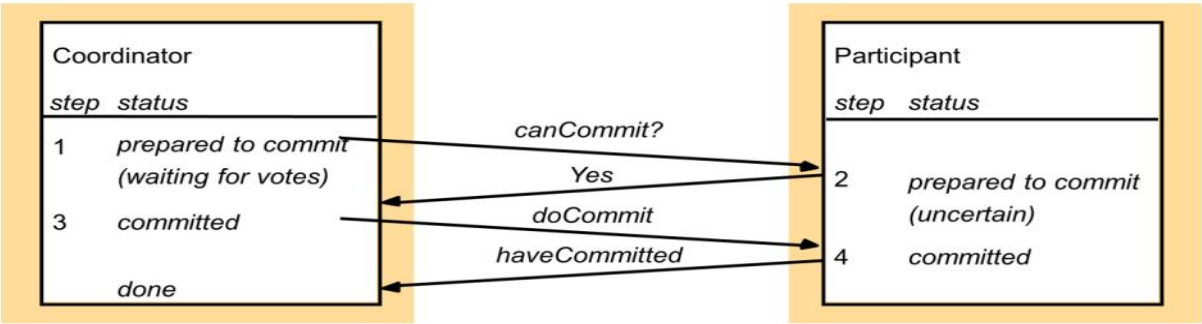
Phase 2 (completion according to outcome of vote):

3. The coordinator collects the votes (including its own).

(a) If there are no failures and all the votes are Yes the coordinator decides to commit the transaction and sends a doCommit request to each of the participants.

(b) Otherwise the coordinator decides to abort the transaction and sends doAbort requests to all participants that voted Yes.

4. Participants that voted Yes are waiting for a doCommit or doAbort request from the coordinator. When a participant receives one of these messages it acts accordingly and in the case of commit, makes a haveCommitted call as confirmation to the coordinator.



Concurrency Control in Distributed Systems:

Concurrency controlling techniques ensure that multiple transactions are executed simultaneously while maintaining the ACID properties of the transactions and serializability in the schedules.
In this chapter, we will study the various approaches for concurrency control.

Locking Based Concurrency Control Protocols

Locking-based concurrency control protocols use the concept of locking data items. A lock is a variable associated with a data item that determines whether read/write operations can be performed on that data item. Generally, a lock compatibility matrix is used which states whether a data item can be locked by two transactions at the same time.
Locking-based concurrency control systems can use either one-phase or two-phase locking protocols.

a) One-phase Locking Protocol

In this method, each transaction locks an item before use and releases the lock as soon as it has finished using it. This locking method provides for maximum concurrency but does not always enforce serializability.

b) Two-phase Locking Protocol

In this method, all locking operations precede the first lock-release or unlock operation. The transaction comprises of two phases. In the first phase, a transaction only acquires all the locks it needs and do not release any lock. This is called the expanding or the growing phase. In the second phase, the transaction releases the locks and cannot request any new locks. This is called the shrinking phase.
Every transaction that follows two-phase locking protocol is guaranteed to be serializable. However, this approach provides low parallelism between two conflicting transactions.

Timestamp Concurrency Control Algorithms

Timestamp-based concurrency control algorithms use a transaction's timestamp to coordinate concurrent access to a data item to ensure serializability. A timestamp is a unique identifier

given by DBMS to a transaction that represents the transaction's start time.

These algorithms ensure that transactions commit in the order dictated by their timestamps. An older transaction should commit before a younger transaction, since the older transaction enters the system before the younger one.

Timestamp-based concurrency control techniques generate serializable schedules such that the equivalent serial schedule is arranged in order of the age of the participating transactions. Some of timestamp-based concurrency control algorithms are –

- Basic timestamp ordering algorithm.
- Conservative timestamp ordering algorithm.
- Mult version algorithm based upon timestamp ordering.

Timestamp based ordering follow three rules to enforce serializability –

- **Access Rule** − When two transactions try to access the same data item simultaneously, for conflicting operations, priority is given to the older transaction. This causes the younger transaction to wait for the older transaction to commit first.
- **Late Transaction Rule** − If a younger transaction has written a data item, then an older transaction is not allowed to read or write that data item. This rule prevents the older transaction from committing after the younger transaction has already committed.
- **Younger Transaction Rule** − A younger transaction can read or write a data item that has already been written by an older transaction.

Optimistic Concurrency Control Algorithm

In systems with low conflict rates, the task of validating every transaction for serializability may lower performance. In these cases, the test for serializability is postponed to just before commit. Since the conflict rate is low, the probability of aborting transactions which are not serializable is also low. This approach is called optimistic concurrency control technique.
In this approach, a transaction's life cycle is divided into the following three phases –

- Execution Phase − A transaction fetches data items to memory and performs operations upon them.

- Validation Phase − A transaction performs checks to ensure that committing its changes to the database passes serializability test.
- Commit Phase − A transaction writes back modified data item in memory to the disk.

This algorithm uses three rules to enforce serializability in validation phase –

Rule 1 − Given two transactions Ti and Tj, if Ti is reading the data item which Tj is writing, then Ti's execution phase cannot overlap with Tj's commit phase. Tj can commit only after Ti has finished execution.

Rule 2 − Given two transactions Ti and Tj, if Ti is writing the data item that Tj is reading, then Ti's commit phase cannot overlap with Tj's execution phase. Tj can start executing only after Ti has already committed.

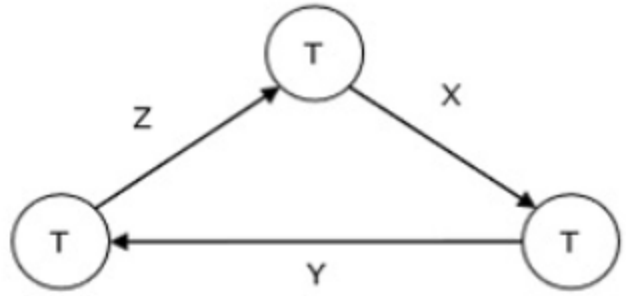
Rule 3 − Given two transactions Ti and Tj, if Ti is writing the data item which Tj is also writing, then Ti's commit phase cannot overlap with Tj's commit phase. Tj can start to commit only after Ti has already committed.

Distributed Dead Locks:

What are Deadlocks?

Deadlock is a state of a database system having two or more transactions, when each transaction is waiting for a data item that is being locked by some other transaction. A deadlock can be indicated by a cycle in the wait-for-graph. This is a directed graph in which the vertices denote transactions and the edges denote waits for data items.

For example, in the following wait-for-graph, transaction T1 is waiting for data item X which is locked by T3. T3 is waiting for Y which is locked by T2 and T2 is waiting for Z which is locked by T1. Hence, a waiting cycle is formed, and none of the transactions can proceed executing.



Deadlock Handling in Distributed Systems

Transaction processing in a distributed database system is also distributed, i.e. the same transaction may be processing at more than one site. The two main deadlock handling concerns in a distributed database system that are not present in a centralized system are transaction location and transaction control. Once these concerns are addressed, deadlocks are handled through any of deadlock prevention, deadlock avoidance or deadlock detection and removal.

Transaction Location

Transactions in a distributed database system are processed in multiple sites and use data items in multiple sites. The amount of data processing is not uniformly distributed among these sites. The time period of processing also varies.

Transaction Control

Transaction control is concerned with designating and controlling the sites required for processing a transaction in a distributed database system. There are many options regarding the choice of where to process the transaction and how to designate the centre of control, like:

- One server may be selected as the centre of control.
- The centre of control may travel from one server to another.
- The responsibility of controlling may be shared by a number of servers.

1) Distributed Deadlock Prevention

In distributed deadlock prevention approach, a transaction should acquire all the locks before starting to execute. This prevents deadlocks.

The site where the transaction enters is designated as the controlling site. The controlling site sends messages to the sites where the data items are located to lock the items. Then it waits for confirmation. When all the sites have confirmed that they have locked the data items, transaction

starts. If any site or communication link fails, the transaction has to wait until they have been repaired.

Though the implementation is simple, this approach has some drawbacks −

- Pre-acquisition of locks requires a long time for communication delays. This increases the time required for transaction.
- In case of site or link failure, a transaction has to wait for a long time so that the sites recover. Meanwhile, in the running sites, the items are locked. This may prevent other transactions from executing.

2) Distributed Deadlock Avoidance

Distributed deadlock avoidance handles deadlock prior to occurrence. Additionally, in distributed systems, transaction location and transaction control issues needs to be addressed. Due to the distributed nature of the transaction, the following conflicts may occur –

- Conflict between two transactions in the same site.
- Conflict between two transactions in different sites.

Let us assume that there are two transactions, T1 and T2. T1 arrives at Site P and tries to lock a data item which is already locked by T2 at that site. Hence, there is a conflict at Site P. The algorithms are as follows –

- **Distributed Wound-Die**
  - If T1 is older than T2, T1 is allowed to wait. T1 can resume execution after Site P receives a message that T2 has either committed or aborted successfully at all sites.
  - If T1 is younger than T2, T1 is aborted. The concurrency control at Site P sends a message to all sites where T1 has visited to abort T1. The controlling site notifies the user when T1 has been successfully aborted in all the sites.
- **Distributed Wait-Wait**
  - If T1 is older than T2, T2 needs to be aborted. If T2 is active at Site P, Site P aborts and rolls back T2 and then broadcasts this message to other relevant sites. If T2 has left Site P but is active at Site Q, Site P broadcasts that T2 has been aborted; Site L then aborts and rolls back T2 and sends this message to all sites.
  - If T1 is younger than T1, T1 is allowed to wait. T1 can resume execution after Site P receives a message that T2 has completed processing.

3) Distributed Deadlock Detection

Deadlocks are allowed to occur and are removed if detected. The system does not perform any checks when a transaction places a lock request. For implementation, global wait- for-graphs are created. Existence of a cycle in the global wait-for-graph indicates deadlocks. However, it is difficult to spot deadlocks since transaction waits for resources across the network.

Another tool used for deadlock handling is a deadlock detector. In a centralized system, there is one deadlock detector. In a distributed system, there can be more than one deadlock detectors. A deadlock detector can find deadlocks for the sites under its control. There are three alternatives for deadlock detection in a distributed system, namely.

- Centralized Deadlock Detector − One site is designated as the central deadlock detector.
- Hierarchical Deadlock Detector − A number of deadlock detectors are arranged in hierarchy.
- Distributed Deadlock Detector − All the sites participate in detecting deadlocks and removing them.

Transaction Recovery:

Transaction recovery is done to eliminate the adverse effects of faulty transactions rather than to recover from a failure. Faulty transactions include all transactions that have changed the database into undesired state and the transactions that have used values written by the faulty transactions.

Types of Transaction Recovery

Recovery information is divided into two types:

- Undo (or Rollback) Operations
- Redo (or Cache Restore) Operations

Ingres performs both online and offline recovery, as described in Recovery Modes (see page Recovery Modes).

Undo Operation:

Undo or transaction backout recovery is performed by the DBMS Server. For example, when a transaction is aborted, transaction log file information is used to roll back all related updates. The

DBMS Server writes the Compensation Log Records (CLRs) to record a history of the actions taken during undo operations.

Redo Operation

A Redo recovery operation is database-oriented. Redo recovery is performed after a server or an installation fails. Its main purpose is to recover the contents of the DMF cached data pages that are lost when a fast-commit server fails. Redo recovery is performed by the recovery process. Redo recovery precedes undo recovery.

Redo Operation in a Cluster Environment

In an Ingres cluster environment where all nodes are active, the local recovery server performs transaction redo/undo for a failed DBMS server on its node, just like in the non-cluster case. The difference in a cluster installation is that if the recovery process (RCP) dies on one node, either because of an Ingres failure, or a general failure of the hardware, an RCP on another node will take responsibility for cleaning up transactions for the failed nodes.